

CMSC3621 Data Structures/Algorithms Lab

Lab Assignment 8

Overview:

This assignment is for the students to practice implementing Depth-First Search and Breadth-First Search in Graph.

```
void Graph::dfs() {
    // 1. Initialize all vertices to unvisited

    // 2. For each vertex v in the vertex set V
    // (hint: you can use a for loop with numOfVertices to go through each vertex in the
    graph)

    //     If the vertex is not visited (use the index i of the for loop to check the bool
    vector visited whether it is false)
    //     call dfs_util() to go deeper in that vertex
    //     to call dfs_util(), you will pass two parameters, the index i and the bool vector
    visited
}

void Graph::dfs_util(int v, vector<bool> &visited) {

    // 1. Declare a stack named s (use the STL stack of int type)
    // 2. Push the vertex v to the top of the stack

    // 3. While the stack s is not empty (while loop iteration)
    //     in each iteration, get the top item of the stack s as the vertex v (hint: use
    top() in STL stack)
    //     pop the top item in the stack

    //     check if the vertex v is not visited, if it is not visited
    //     mark the vertex v as visited in the bool vector visited
    //     print out the result (use print_letter() to print out the vertex v, it
    will print out the capital letter instead of the index number)

    //     use a for loop to go through all the adjacent vertices of the vertex v
    //     (as AdjList is an array of STL list, you need to use adjList[v] to access the
    list of that vertex v)
    //     (you will also need to use iterator in the list, we did it in lab6)
    //     (you can use auto keyword for auto-define the type of the iterator it, first
    you will start with adjList[v].begin(),
    //     as long as it has not reached adjList[v].end(), the loop will continue, it
    will increase one step in every iteration)

    //     check if the vertex that the iterator it is pointing to is not visited (as
    iterator it is also a pointer,
    //     to retrieve the vertex that it is pointing to, we can use dereference
    operator, i.e. check whether visited[*it] is false)
    //     push the vertex to the stack if it is not yet visited so that we can
    continue to go deeper
}
```

```

void Graph::bfs() {
    // 1. Initialize all vertices to unvisited
    // 2. Declare a queue named q (use the STL queue of int type)

    // 3. For each vertex v in the vertex set V
    // (hint: you can use a for loop with numOfVertices to go through each vertex in the
    graph)

        //      If the vertex is not visited (use the index i of the for loop to check the bool
        vector visited whether it is false)
        //          enqueue the vertex to the queue (in STL queue, the enqueue function is also
        named push(), which will push the item to the back of the queue)
        //          mark the vertex as visited in the bool vector visited
        //          print out the result (use print_letter() to print out the vertex, it will
        print out the capital letter instead of the index number)

    // 4. While the queue q is not empty (while loop iteration)
    //      in each iteration, get the first item of the queue q as the vertex v (hint:
    use front() in STL front)
    //      dequeue the first item in the stack (in STL queue, the dequeue function is
    also named pop(), which will remove the first item in the queue)

    //          use a for loop to go through all the adjacent vertices of the vertex v
    //          (as AdjList is an array of STL list, you need to use adjList[v] to access the
    list of that vertex v)
    //          (you will also need to use iterator in the list, we did it in lab6)
    //          (you can use auto keyword for auto-define the type of the iterator it, first
    you will start with adjList[v].begin(),
    //          as long as it has not reached adjList[v].end(), the loop will continue, it
    will increase one step in every iteration)

    //          check if the vertex that the iterator it is pointing to is not visited (as
    iterator it is also a pointer,
    //          to retrieve the vertex that it is pointing to, we can use dereference
    operator, i.e. check whether visited[*it] is false)
    //          enqueue the vertex to the queue
    //          mark the vertex as visited in the bool vector visited
    //          print out the result (use print_letter() to print out the vertex, it
    will print out the capital letter instead of the index number)
}

```

Requirements:

- 1) Follow the comments of the functions to complete the code of `dfs()`, `dfs_util()` and `bfs()`.
- 2) When you finish the `dfs()`, `dfs_util()` and `bfs()` methods, run the code in `main.cpp`, but do not change the testing code in `main.cpp`.

Submission

- 1) graph.h //Comments are needed to illustrate how do you implement the code
- 2) A report (a PDF file), in the report you need to provide screenshots and descriptions about why your solution is correct according to the output (you must show your understanding about BFS and DFS to earn full points).

Important requirements

- 1) Don't include any other files (that are not aforementioned).
- 2) Don't modify anything in the existing files / project, otherwise your code may not be able to run in the grader's environment.
- 3) You only can write your code at certain places which are indicated by the existing comments.
- 4) Any violation against the rules above will lead to extra penalty. If you missed the report, your submission will not be evaluated.

Correct output of the program: (if we use push_front() in addEdge() in the Graph)

```
Iterative Depth-first traversal Result of the Graph g:
A B E G F C H D

Breadth-first traversal Result of the Graph g:
A G D B E F C H
```

Correct output of the program: (if we use push_back() in addEdge() in the Graph)

```
Iterative Depth-first traversal Result of the Graph g:
A G E B F D C H

Breadth-first traversal Result of the Graph g:
A B D G E F C H
```

Rubric

- 1) There are three incomplete functions in total. Partial credits may be given in terms of to what degree you can complete them.
 - 1.1) dfs_util(): 25 pts
 - 1.2) dfs(): 25 pts
 - 1.3) bfs(): 30 pts
- 2) Report: 20 pts